

华南师范大学

South China Normal University

本科毕业论文

基于 ROS 的多机器人协同搜索系统

学 生 姓 名： 罗隽石
学 号： 20212131077
专 业： 计算机科学与技术
所 在 院 系： 计算机学院
导师姓名及职称： 冯刚 副教授

2025 年 3 月 26 日

基于 ROS 的多机器人协同搜索系统

专业名称： 计算机科学与技术

申请者： 罗隽石

导师： 冯刚

摘 要

随着智能技术和机器人技术的迅速发展，多机器人协同系统在多个领域逐渐展现出巨大的潜力。传统的单一机器人执行搜索任务面临着覆盖范围有限、响应速度慢和适应能力差等问题，难以应对动态变化和复杂环境下的高效协作需求。为了解决这些问题，研究者们开始探索多机器人协同工作的方式，通过机器人之间的信息共享与协作，能够大幅度提高任务的完成效率与系统的鲁棒性。基于这一背景，本论文将介绍一种多机器人协同搜索系统，旨在通过多个机器人之间的紧密配合与智能决策，提升搜索任务的执行效果，探索多机器人系统的工作流程以及特点。

本论文研究了多机器人协同搜索系统的设计与实现，旨在通过多机器人协作提高搜索任务的效率与可靠性。在该系统中，多个机器人通过共享感知信息和协调行动，能够有效地覆盖更广泛的搜索区域，同时解决了单一机器人在复杂环境中执行任务时可能遇到的局限性。我们提出了一种基于 ROS2 的分布式通信框架，采用自动路径规划算法、目标检测技术和协同策略，以实现机器人间的高效合作。系统包括多机器人导航、地图融合、障碍物避让、AI 目标识别与坐标解算、以及信息融合等模块。实验结果表明，系统在仿真环境下能稳定运行，显著提高了搜索效率，展现了完善合理的流程。最后，本文还探讨了未来该系统在家用、工业、军事等场景中的应用前景，并提出了进一步优化与改进的方向。

关键词： 多机器人;ROS2; 机器人协同; 目标检测; 定位

ROS-based Multi-Robot Collaborative Search System

Major: Computer Science and Technology

Name: Luo Juanshi

Supervisor: Feng Gang

ABSTRACT

South China Normal University is an institution of higher education with a long history and a rich legacy. Situated in Guangzhou, the open metropolis in South China, South China Normal University is imbued with Lingnan's pioneering and pragmatic spirits. It has developed a tradition of elegant simplicity and fostered a strong learning environment. In the past seventy years or so, South China Normal University has preserved its characteristics of teacher education and has been devoted to cultivating talents with moral integrity, independent thinking, innovative ability and sense of social responsibility.

In the new century in which the institutions of higher education are getting increasingly involved in the social and economic development, South China Normal University has adopted an international view on education. Having broadened its horizon on the key issue of cultivating talents, it has made its greatest efforts to create a congenial and harmonious environment for both teaching and academic research and has fostered a rich variety of campus culture. It aims to build itself into a high-level comprehensive teaching and research-oriented university with open distinctive features.

KEY WORDS: Multi-robot; ROS2; Robot Collaboration; Target Detection; Localization

目 录

摘 要	I
ABSTRACT	III
第一章 绪论	1
1.1 工程概述	1
1.1.1 工程背景和意义	1
1.1.2 工程基本目标	2
1.2 研究思路和主要工作	2
1.3 论文结构安排	3
第二章 多机器人系统架构设计	5
2.1 系统开发与运行平台	5
2.1.1 机器人操作系统 ROS	5
2.1.2 Gazebo 机器人仿真平台	7
2.1.3 移动机器人 Turtlebot3	7
2.2 总体架构与流程	8
2.2.1 地图融合模块	9
2.2.2 路径规划模块	9
2.2.3 目标检测模块	10
第三章 目标识别与检测功能设计	11
3.1 目标检测模型的选择	11
3.1.1 yolo 目标检测模型	11
3.2 目标检测模型的训练与调优	12
3.2.1 训练环境搭建	12
3.2.2 训练数据采集和预处理	12
3.2.3 训练与调优	13
3.3 模型集成到 ROS 系统	14
第四章 目标检测与搜索流程设计	17
4.1 目标坐标解算	17
4.1.1 计算目标相对方位	17

4.1.2	计算目标距离	18
4.1.3	解算目标坐标	18
4.2	传感器数据融合优化	19
4.2.1	对于目标检测信息的时间戳调整	20
4.2.2	时间戳队列同步	20
4.3	数据处理模块优化目标坐标	22
4.4	本章小结	23
第五章	路径规划与控制模块设计	25
5.1	模块架构	25
5.2	路径规划模块	26
5.2.1	边界识别与探索区域分配	26
5.2.2	进行路径规划	28
5.2.3	机器人请求路径规划	30
5.3	根据路径控制机器人	30
5.3.1	路径控制算法	31
5.3.2	路径跟随的实现	31
第六章	系统的仿真与测试	33
6.1	仿真环境	33
6.1.1	主要运行环境	33
6.1.2	主要仿真参数	33
6.2	在仿真环境下运行系统	35
6.3	系统测试	38
第七章	总结与展望	41
7.1	总结	41
7.2	展望	41
参考文献		43
致 谢		45

表 目 录

表 2-1	TurtleBot3 Burger 与 Waffle 型号对比	8
表 2-2	系统主要 ROS 包及其功能	8
表 3-1	yolov5s 模型训练参数	14
表 3-2	yolo_result 话题的目标检测结果消息格式	15
表 4-1	LaserScan 话题的激光扫描数据格式	18
表 4-2	mulit_robot_map_updater 节点的话题列表	24
表 6-1	主要运行环境配置	33
表 6-2	Gazebo 物理引擎参数设置	34

图 目 录

图 2-1	Gazebo 机器人仿真平台	7
图 2-2	系统基本架构图	9
图 3-1	Gazebo 中的啤酒罐模型	13
图 3-2	对原始图片进行数据增强	13
图 3-3	测试目标检测模型效果	14
图 3-4	yolov5_ros2 节点与话题示意图	15
图 3-5	错误颜色 (上) 与正确颜色 (下) 的对比	16
图 4-1	目标识别模块运行信息	21
图 4-2	仿真环境中的目标与地图中的目标标记	23
图 4-3	mulit_robot_map_updater 节点的 ROS 结构	24
图 5-1	路径规划与控制模块节点结构	25
图 6-1	仿真房屋环境搭建	34
图 6-2	在仿真房屋中放入机器人	35
图 6-3	地图融合模块	35
图 6-4	目标检测模块运行信息	36
图 6-5	目标搜索与定位显示界面	36
图 6-6	带有目标检测框的图像	37
图 6-7	原始目标坐标点	37
图 6-8	红绿色规划路线	37
图 6-9	数据处理后坐标	37
图 6-10	路径规划模块响应请求	37
图 6-11	路径规划模块完成探索	38
图 6-12	运行完毕得到目标地图	38

第一章 绪论

1.1 工程概述

1.1.1 工程背景和意义

近年来，随着人工智能、物联网、大数据以及边缘计算等前沿技术的迅猛发展，机器人技术正以前所未有的速度渗透进人们的日常生活与工作场景。家用、学校等相对较小且环境复杂的场所对机器人的搜索、定位和决策提出了更高要求，而单个机器人往往因硬件资源、传感器精度和算法实时性等方面的限制，难以独立完成高效的任務。因此，多机器人协同搜索系统通过多台机器人之间的紧密配合与任务分解，有效弥补了单机在覆盖范围和处理速度上的不足，为室内安防、智能巡检以及教育科研等领域提供了值得参考的解决方案。

在这种背景下，ROS（机器人操作系统）凭借其开源、模块化和高度可扩展的特性，成为实现多机器人系统协同作业的重要平台。ROS 不仅提供了丰富的通信机制，如 Topic、Service 和 Action 等标准接口，方便各机器人之间实时数据交换，而且内置了导航、路径规划和障碍物避让等常用算法，为系统开发者节省了大量调试和集成的工作量。实际工程中，诸如 TurtleBot 系列和 Clearpath Robotics 的产品已经验证了基于 ROS 平台构建多机器人协同系统的可行性，这些成熟的技术方案为本系统的研发提供了坚实的实践基础。

此外，近年来以 YOLO（You Only Look Once）为代表的目标检测算法，由于其端到端、实时性强的特点，正在被越来越多的实际项目采用。这种人工智能视觉技术具有轻量级、易部署等特点，可以降低部署的成本，十分适用于各种小型设备。在实际应用中，智慧校园、智能家居以及工业检测等领域已经开始引入这种的视觉识别方案，取得了明显的应用效果。

与此同时，精准的环境感知和地图构建是多机器人协同搜索系统的重要环节。通过整合 SLAM 算法，如 GMapping、Hector SLAM 和 Cartographer，以及激光雷达、RGB-D 摄像头和惯性测量单元（IMU）等多种传感器的数据，系统能够实时

构建并更新高精度的环境地图。这种传感器融合技术在国内外的多项机器人研究项目中已得到广泛应用，有效提升了机器人在复杂室内环境中的自主导航和定位能力。

总的来说，结合以上的技术，可以开发基于 ROS 的多机器人协同搜索系统。通过整合分布式通信架构、成熟的 SLAM 与传感器融合技术以及 YOLO 等高效视觉检测算法，能够在家用、学校等小型环境中实现精准、高效的目标搜索与定位。该系统不仅为提升室内安防、智能巡检等任务的执行效率提供了有效技术支持，也为机器人群体智能和分布式控制的研究探索提供了宝贵的实践经验，预示着在未来智能家居、智慧校园及其他领域中的广阔应用前景。

1.1.2 工程基本目标

多机器人协同搜索系统应具有两个及以上机器人，在仿真环境下使用激光雷达和光学摄像头对未知的环境进行 slam 建图，并且对该环境下的目标进行识别并解算坐标，完成目标在地图上的标记。与此同时，该系统应该具有自动规划路径的功能，包括避障，机器人之间避碰，未知环境探索等能力。

1.2 研究思路和主要工作

本项目将在 Ubuntu20.04 环境下，完成上文提及的基本目标，使用 gazebo 仿真技术对系统进行仿真与验证，主要工作包括以下部分：

1. 研究和设计一个多机器人系统架构方案，能有效地控制和协调系统中机器人之间的行为和功能逻辑关系，及设计多机器人通信架构。
2. 针对多机器人在探索过程中的目标检测问题，设计一种基于 yolo 模型的多机器人目标检测方案并训练调优 yolo 视觉识别模型。
3. 针对多机器人目标搜索定位的问题设计多传感器融合的解决方案，设计模块优化目标定位的效果。
4. 设计路径规划模块并适应多机器人协作场景，包括协同探索，碰撞规避等功能。

1.3 论文结构安排

本论文分为以下七个章节：

第一章：绪论，对工程进行概述，介绍本工程的背景与研究思路，并且说明本论文的结构安排。

第二章：多机器人协同系统架构设计。介绍多机器人协同搜索系统的架构设计与主要模块。

第三章：机器人目标识别与检测功能设计。介绍机器人使用的目标检测模型，以及其训练调优方案。

第四章：目标检测与搜索流程设计。说明多机器人协同搜索系统传感器融合完成目标坐标解算的流程与模块设计。

第五章：路径规划模块设计。说明多机器人协同搜索系统的路径规划模块设计与针对多机器人协作场景进行的优化。

第六章：多机器人协作探索及目标检测实验与分析。搭建仿真环境进行多机器人协作探索及目标检测，在仿真环境中进行自主探索并检测目标物，并对实验结果进行分析总结。

第七章：总结与展望。对本文已完成的研究工作和内容进行总结，分析工作中存在的不足并对后续工作进行了展望。

第二章 多机器人系统架构设计

相较于传统单机器人搜索模式，多机器人协同通过任务分配、资源共享与动态协调，能够有效提升搜索效率、扩大覆盖范围并增强系统容错能力。然而，多机器人协同搜索系统的实现面临诸多问题：机器人间的实时通信、动态环境下的任务分配优化、机器人碰撞规避以及系统可扩展性设计等问题，均需通过合理的系统架构予以支撑。本章基于机器人操作系统 `ros2-galactic` 的分布式框架，提出一种多机器人协同搜索系统架构，包括机器人部署模块，地图融合模块，目标检测模块，目标位置解算模块，路径规划模块。该架构旨在实现机器人资源的高效调度、环境信息的融合共享以及优化协同搜索行为，为后续章节中目标检测算法、路径规划策略及仿真实验验证提供基础框架。

2.1 系统开发与运行平台

2.1.1 机器人操作系统 ROS

ROS (Robot Operating System) 是当下最受欢迎与广泛使用的开源机器人操作系统。它源于斯坦福大学人工智能实验室 STAIR 项目创建的软件系统原型，2007 年 Willow Garage 公司与之合作，在积累了大量研究人员的科学研究成果后逐渐形成了 ROS 核心概念和基础软件架构。ROS 的分布式架构、模块化设计及丰富的工具链，为多机器人协同系统提供了高效开发基础。ROS 具有以下几个特性：

- 开源性与社区生态

ROS 遵循 BSD 开源协议，允许个人及企业免费使用与二次开发。其庞大的社区生态（如 ROS Wiki、GitHub 仓库）提供了海量开源功能包，涵盖传感器驱动、SLAM、路径规划等核心模块，极大降低了机器人系统的开发门槛。

- 分布式架构与松耦合设计

ROS 系统的通信架构允许建立多个节点，通过话题和订阅等方式来管理机器人通信，十分适用于模块化开发，有利于调整增加机器人的功能，使系统

具有很好的扩展性和鲁棒性。

- 多语言与跨平台支持

ROS 支持 C++、Python 等主流编程语言，其中 Python 语法简洁，软件包支持丰富，具有开发效率上的优势。ROS2 Galactic 版本全面适配 Python 3.8+，并强化了对嵌入式平台（如树莓派）、实时操作系统（RTOS）及 Windows 系统的兼容性。

- 第三方工具与算法集成

ROS 可无缝集成 OpenCV、PCL（点云库）、TensorFlow 等第三方库，同时支持 Gazebo 仿真环境、RViz 可视化工具及 Qt 人机界面框架。ROS2 Galactic 进一步优化了与 DDS 中间件（如 Cyclone DDS）的兼容性，为多机器人协同提供低延迟通信保障

- 选用 ROS2 Galactic

尽管 ROS1 在单机器人系统中表现优异，但其通信实时性不足、网络依赖性强等缺陷限制了多机器人协同场景的应用。ROS2 Galactic 作为长期支持（LTS）版本，针对上述问题进行了全面革新：

- 去中心化通信：基于 DDS 协议实现多机器人动态组网，避免单点故障，支持定制 QoS 策略（如低延迟、高可靠性传输）；
- 实时性与安全性：通过生命周期节点管理、线程安全接口及加密通信机制，提升系统稳定性；
- Python 开发优势：ROS2 Galactic 提供成熟的 rclpy 库，结合 Python 的轻量级代码与丰富算法库（如 NumPy、PyTorch），可快速实现任务分配、动态路径规划等协同逻辑，显著提升开发效率。

综上，ROS2 Galactic 凭借其分布式架构、强实时通信及对 Python 生态的深度支持，成为构建高可靠、可扩展多机器人协同搜索系统的理想技术基座。

2.1.2 Gazebo 机器人仿真平台

Gazebo 是一款机器人仿真软件平台，具有高性能的 ODE 物理引擎，提供高质量的三维图形并支持多种编程语言。Gazebo 支持多种机器人传感器模型（如激光传感器、IMU、Kinect 等），提供多种机器人模型（如 PR2、Turtlebot、机械臂等）并支持定制私人机器人模型，并且能够对真实环境中物理实体进行建模，为开发人员编写机器人应用程序以及算法快速测试提供了良好的仿真平台。

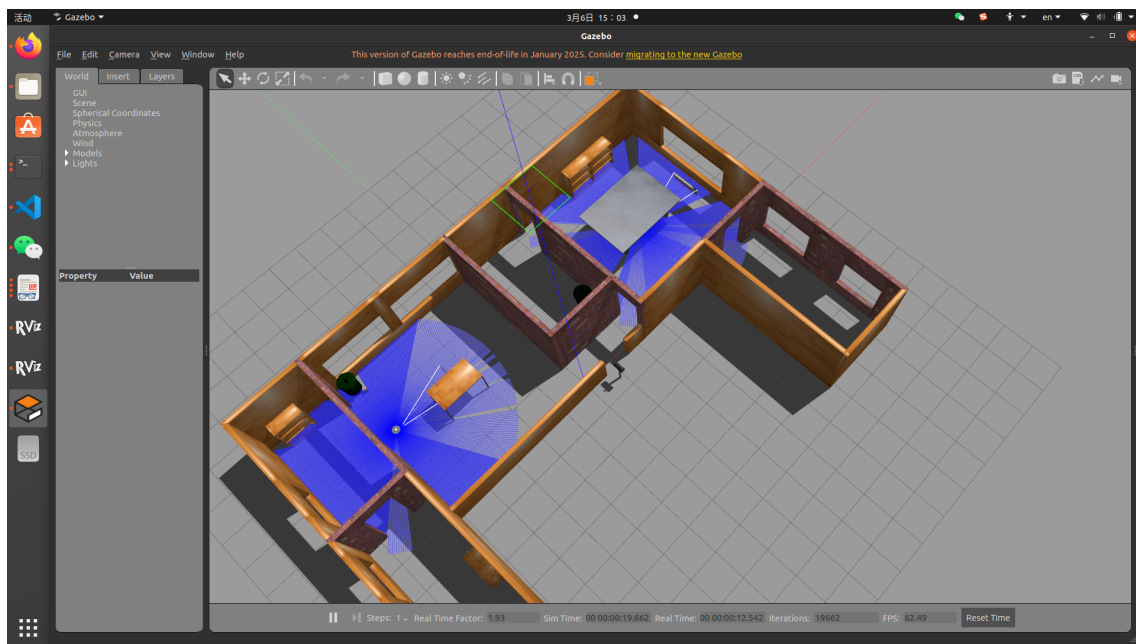


图 2-1 Gazebo 机器人仿真平台

2.1.3 移动机器人 Turtlebot3

移动机器人是多机器人系统中最重要执行本体，是一个集环境感知、任务规划、动作执行等功能于一体的硬件系统。TurtleBot3 是第三代基于 ROS 的开源移动机器人平台，由 ROBOTIS 公司设计并开源，旨在为教育、科研、产品原型开发提供低成本、高灵活性的机器人开发工具。TurtleBot3 的尺寸仅为前代产品的 1/4（如 Burger 型号尺寸为 138×178×192mm），重量仅 1kg，便于携带。其价格远低于同类 SLAM 机器人，适合教育场景的规模化应用。并且作为 ROS 官方支持的机器人平台，TurtleBot3 原生兼容 ROS 的导航栈（如 Gmapping、Cartographer）

和工具链（如 Gazebo 仿真、RViz 可视化），使用效率更高。

TurtleBot3 提供 Burger 和 Waffle Pi 两种主流型号，主要区别如下：

机器人型号	TurtleBot3 Burger	TurtleBot3 Waffle
最大平移速度	0.22 m/s	0.26 m/s
最大角速度	2.84 rad/s	1.82 rad/s
最大负载	15 kg	30 kg
尺寸 (长 × 宽 × 高)	138 mm × 178 mm × 192 mm	281 mm × 306 mm × 141 mm
自重	~1 kg	~1.8 kg
预计工作时间	2.5 小时	2 小时
控制单元	树莓派 3	Intel Joule 570x
传感器配置	360° 激光雷达 (LDS-01)；无摄像头	360° 激光雷达 (LDS-01)；内置摄像头

表 2-1 TurtleBot3 Burger 与 Waffle 型号对比

为了提高环境感知能力，同时允许机器人通过光学摄像头识别搜索目标，本项目选用 Turtlebot3 Waffle 机器人。

2.2 总体架构与流程

多机器人目标搜索系统利用 ROS2 将各个资源与模块相连接，使得各个节点之间具有松耦合和模块化的特性，保证了系统的可扩展性、稳定性和灵活性，系统工作的总体流程示意图如图 2-2 所示。

每个 ROS2 包分别负责不同的功能，包括机器人控制、仿真环境搭建、目标检测等，系统主要由以下 ROS2 包构成，根据功能构成多个模块：

ROS 包名	主要功能
merge_map	地图融合
multi_robot_exploration	在仿真环境中部署机器人
path_follow	根据规划路径控制 turtlebot 机器人的运动
target_detect	目标搜索与坐标解算
turtlebot3_gazebo	仿真环境搭建与配置
yolov5_ros2	yolo 模型目标识别

表 2-2 系统主要 ROS 包及其功能

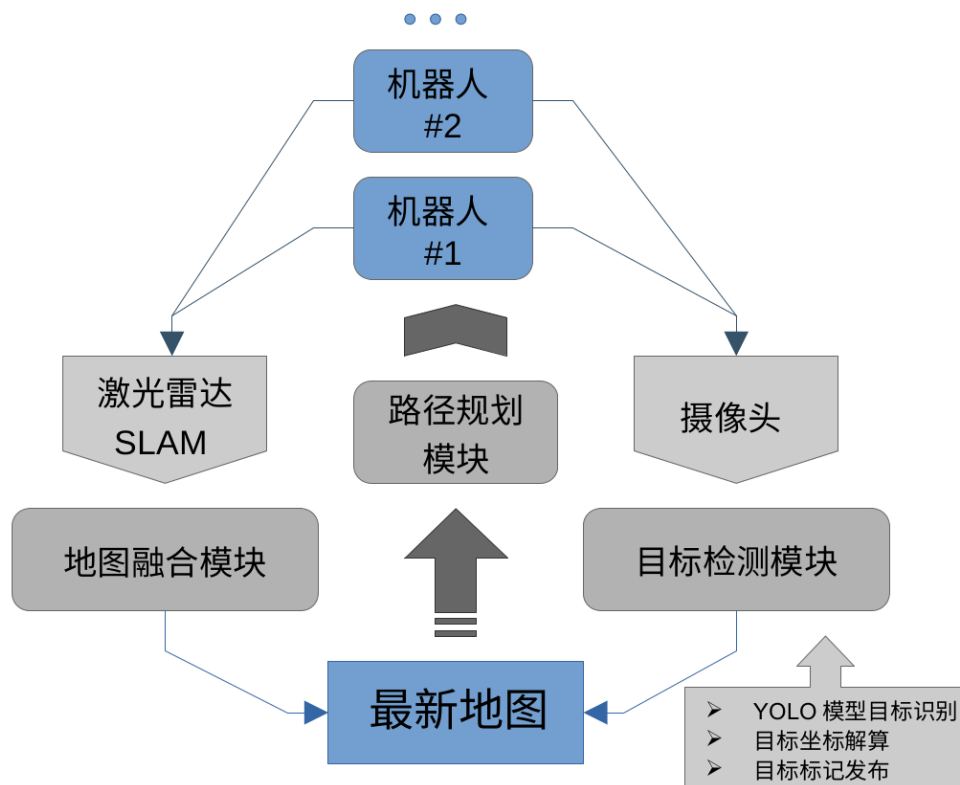


图 2-2 系统基本架构图

2.2.1 地图融合模块

全局地图融合是多机器人协作探索环境和地图构建系统中的关键部分之一。全局地图融合模块首先加载多机器人初始相对位姿参数，实时检测多机器人局部地图信息并对地图数据接收。根据当前系统中的相关参数以及多机器人实时构建的局部地图确定全局地图的尺寸与位姿，最后根据多张局部地图信息进行全局地图融合。

2.2.2 路径规划模块

路径规划模块将根据机器人 SLAM 得到的地图进行障碍地图的生成，并基于此进行路径规划以完成对未知环境的探索。使用了 A* 等算法进行动态路径规划，可以同时为多个机器人规划路径，为了适用与多机器人搜索的要求，路径规划时会考虑多机器人之间的避碰，并且提高搜索效率。规划模块会将路径话题发布到包 `path_follow` 进行解算后最终控制机器人的运动。

2.2.3 目标检测模块

目标检测模块使用了机器人上的摄像头模块，利用 yolov5s 人工智能视觉模型对目标进行识别，并且将识别的数据与激光雷达的距离数据进行融合，最后结合机器人位姿坐标进行目标坐标的标记与解算。为了提高效率与精确度，传感器数据融合过程使用了时间同步策略，并且对解算得到的原始坐标进行后期数据处理，使用聚类分析等数据处理技术对目标坐标进行推理优化。

第三章 目标识别与检测功能设计

为了实现多机器人的搜索功能，机器人必须具备准确的目标识别能力，以便在复杂环境中快速捕捉和定位感兴趣的目标。与传统的视觉方案相比，传统方法往往依赖于手工设计的特征提取算法，容易受到光照变化、遮挡和噪声的影响；而神经网络目标检测模型则能通过大量数据学习到更具泛化性的特征，在多变的场景中表现出更高的鲁棒性和识别准确率。因此，本项目选择采用基于神经网络的目标检测模型对目标进行识别。为此，主要工作包括对不同模型进行筛选和评估，确定最适合当前应用场景的方案；随后在大规模数据集上对模型进行训练，并结合迁移学习等技术进行参数调优；最后，还需要将训练好的模型与 ROS 系统进行适配和集成，确保目标检测模块能够实时、高效地与多机器人搜索系统中的其他模块协同工作。

3.1 目标检测模型的选择

3.1.1 yolo 目标检测模型

YOLO（You Only Look Once）是一类基于卷积神经网络（CNN）的目标检测算法，它通过一次前向传播同时实现目标定位与分类，从而大幅提高检测速度。YOLOv5s 是该系列中较为轻量化的版本，其网络结构设计精简，参数量较少，在保证较高检测准确率的同时具有极快的推理速度。对于多机器人协同搜索系统来说，每个机器人都需要在动态环境中快速、准确地检测并定位目标，以便实时更新全局地图和调整搜索策略。YOLOv5s 正因其低计算负载和高实时性，非常适合在资源受限的机器人平台上运行，并且易于与 ROS 系统集成，为多机器人系统提供高效的目标识别支持。

3.2 目标检测模型的训练与调优

3.2.1 训练环境搭建

本项目在一台搭载 NVIDIA GeForce RTX 3060 (Laptop 版) 的笔记本电脑上构建了一个训练平台, 从 Ultralytics 的 GitHub 仓库中克隆了 YOLOv5 官方代码库。该代码库不仅包含了经过大量实践验证的模型架构和训练脚本, 还提供了详尽的文档和示例配置文件。这些资源提供了一个成熟、可靠的基础平台, 使得后续的模式训练与调优能够更加高效和精准。此外, 官方代码库的持续更新和广泛的社区支持, 也为模型优化提供了丰富的参考和技术保障。

为了保证实验环境的隔离性和依赖库的一致性, 采用了 Anaconda 创建独立的 Python 虚拟环境。选用 Python 3.10 作为开发语言, 确保了与 YOLOv5 代码及相关依赖的兼容性。在虚拟环境中, 通过官方提供的 requirements.txt 文件安装了包括 PyTorch、NumPy、OpenCV 等在内的所有必要依赖。此举不仅避免了系统中不同项目之间的依赖冲突, 还提升了实验的可重复性和调试效率。并且依据所使用的 PyTorch 版本及其与 CUDA 的兼容要求, 安装了合适版本的 CUDA 工具包与 cuDNN 库。正确配置的 CUDA 与 cuDNN 能够显著提高大规模矩阵运算的速度, 缩短训练时间, 从而使得 YOLOv5 模型在处理复杂数据时表现出更高的效率和稳定性。这一步骤对整体训练过程很重要, 因为它直接影响到模型的迭代速度和调优效果。

3.2.2 训练数据采集和预处理

将 gazebo 仿真环境下的啤酒罐 (图 3-1) 设定为机器人的搜索目标。为采集具有更多信息的数据, 在仿真环境中从不同角度和距离录制目标的视频, 然后使用 python 脚本将该视频按帧率分割为一组图片序列, 获取原始训练数据。使用 labeling 数据标注工具按照 yolo 格式对图片进行标注, 此时数据集的总规模为 90 张图片。然后对数据集进行数据增强, 数据增强的手段包括放缩、增加噪声、改变角度、调整曝光、改变色调等 (图 3-2), 最终将训练集的规模扩展到 780 张图片。

此时将数据集划分训练集 (train)、验证集 (val) 和测试集 (test)，最后构建出完整的训练数据集。



图 3-1 Gazebo 中的啤酒罐模型

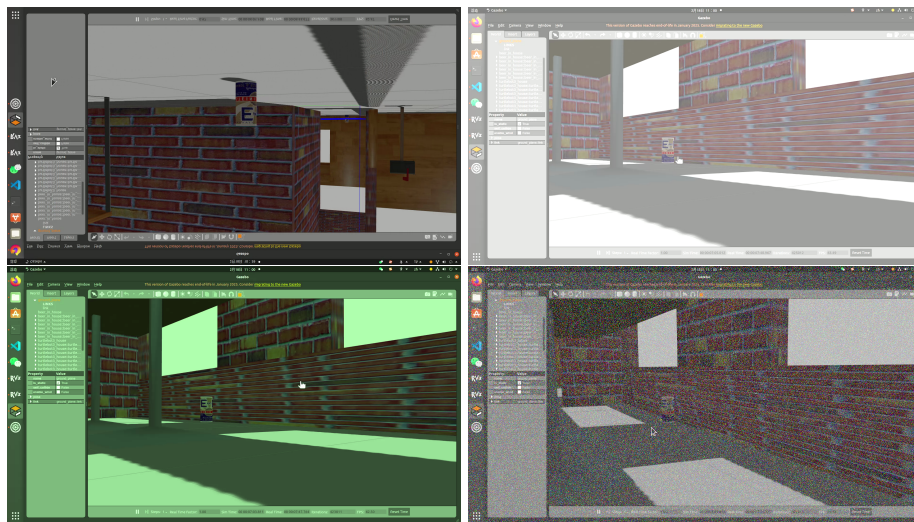


图 3-2 对原始图片进行数据增强

3.2.3 训练与调优

完成数据集的处理后，为适应硬件平台的性能条件和优化模型训练效果, 调整训练 yolov5s 模型的训练参数，完成训练后得到最佳权重文件 **best.pt**，使用的关键

训练参数如下表所示：

训练参数	值
weights	yolov5s.pt
epochs	200
batch_size	4
imgsz	640
workers	2
optimizer	SGD

表 3-1 yolov5s 模型训练参数

使用模型对含有目标的图像进行推理检测，以测试模型对目标的识别效果，评估模型对目标的检测能力（图 3-3）。直到模型对目标有明显且稳定的识别效果，并且需要关注每个目标的置信度，为后期过滤可能存在的误报做准备。



图 3-3 测试目标检测模型效果

3.3 模型集成到 ROS 系统

为了将目标检测模型部署到多机器人搜索系统中，需要将模型的推理过程结合到 ROS 系统中，订阅两个机器人的摄像头图像话题，并且在完成检测后将检测结果发布出去。为此需要建立 ROS 包 yolov5_ros2，它将会同时接收所有机器人摄像头的图像，使用训练好的 yolov5s 模型进行推理检测，最后将推理完成的位置数据和标记后的图像分别发布到话题 /yolo_result 和 /yolo_img 中。

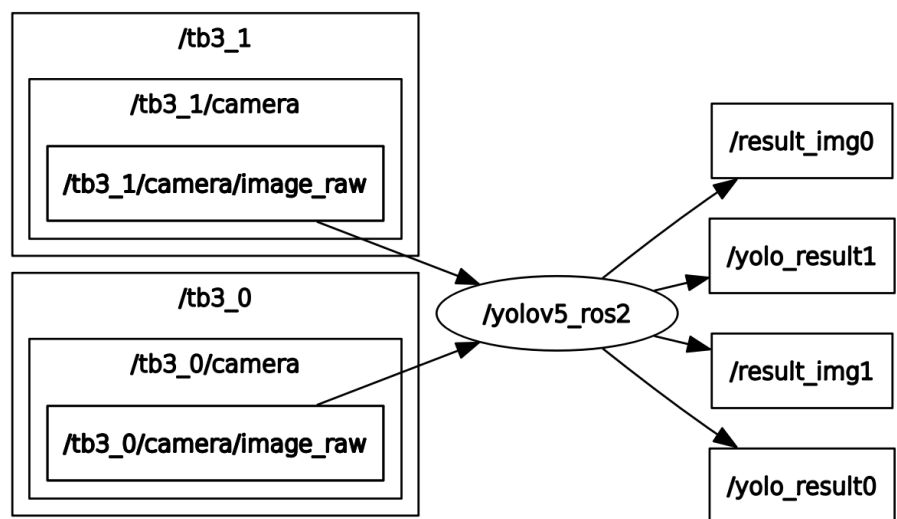


图 3-4 yolov5_ros2 节点与话题示意图

/yolo_result 话题使用了 Detection2DArray 的消息类型，其中包含多类数据，话题中含有的数据及其标签如下表（表 3-2），这些数据将反应目标在图像中的位置，置信度和目标框选范围等。

表 3-2 yolo_result 话题的目标检测结果消息格式

消息字段	含义
header.frame_id	传感器坐标系，如 camera_<robot_id>
header.stamp	图像时间戳，与原始图像一致
detections[]	检测到的目标列表，每个元素为 Detection2D
detections[].id	目标类别名称，如”person”、”car”
detections[].bbox.center.x	目标中心 x 坐标
detections[].bbox.center.y	目标中心 y 坐标
detections[].bbox.center.position.x	目标中心 x 坐标（Galactic 及以上）
detections[].bbox.center.position.y	目标中心 y 坐标（Galactic 及以上）
detections[].bbox.size_x	目标包围框宽度
detections[].bbox.size_y	目标包围框高度
detections[].results[]	目标分类置信度信息列表
detections[].results[].hypothesis.class_id	目标类别 ID
detections[].results[].hypothesis.score	目标置信度分数（[0, 1]）

值得注意的是，其中的 header.stamp 并没有使用 yolo 推理完成后的发布数据的时间戳，而是使用了原始图像获取时的时间戳，这是考虑到图像识别数据主要目的是定位目标位置，因此需要和其他的传感器数据例如激光雷达、位置传感器等进行融合，因此需要保持良好的同步性以提高目标检测的精度，以规避推理、

传输、计算过程带来的延迟导致坐标偏移的问题。

而 `yolo_img` 话题则包含了原始图像完成推理后被标记目标框的图像，便于直观查看目标检测的效果。值得注意的是，由于 `yolo` 和 `OpenCV` 的默认图像格式不同，`yolo` 使用的 GBR 顺序的颜色通道而 `OpenCV` 模型使用 RGB，因此必须进行适当的转换，否则输出的图像会出现明显的颜色错误（图 3-5）。

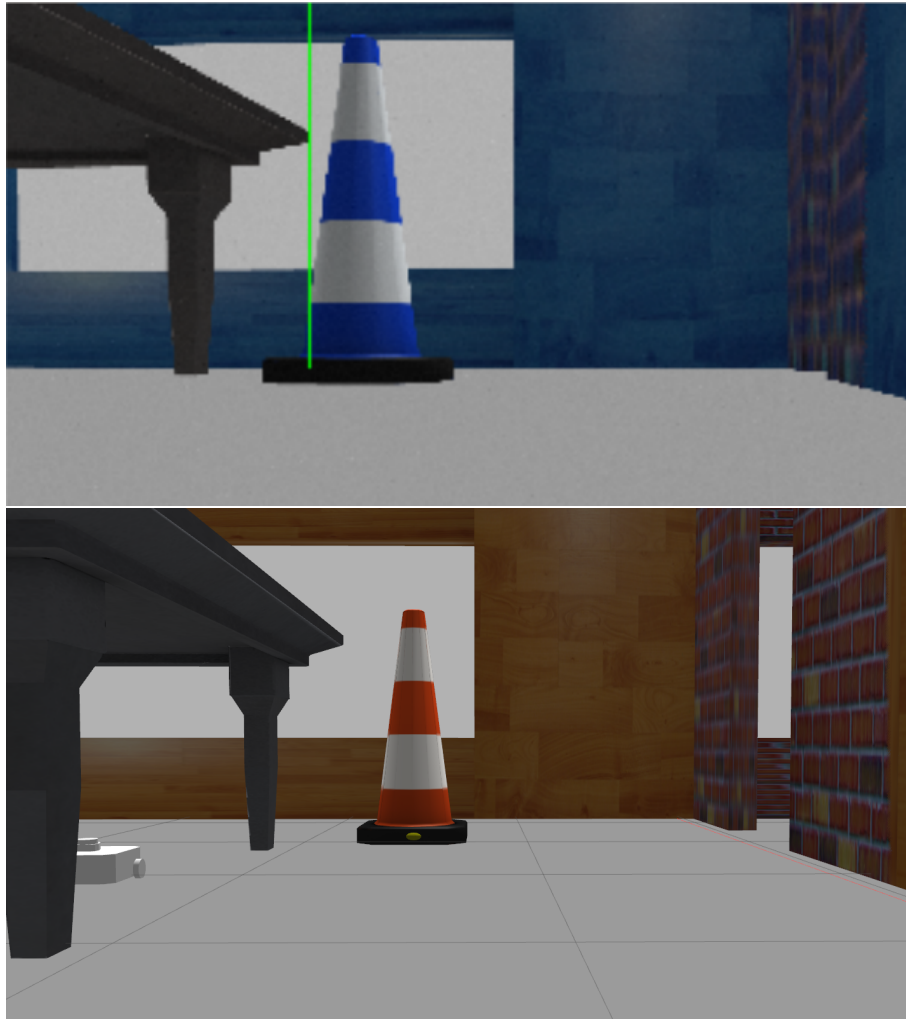


图 3-5 错误颜色 (上) 与正确颜色 (下) 的对比

第四章 目标检测与搜索流程设计

在多机器人协同目标搜索系统中，准确确定目标的位置是核心需求之一。当机器人通过视觉检测识别到目标后，需要结合多种传感器的数据，对目标在地图中的具体坐标进行计算。这一过程中，不仅涉及数据融合技术，还需要在精确度、计算效率和系统稳定性之间找到平衡，以确保机器人能够快速、准确地感知目标位置并做出相应决策。因此，本章将详细介绍系统在目标检测与搜索流程中的设计方案，包括目标定位方法、传感器数据融合优化策略、以及数据处理方案，以提升系统的整体性能和可靠性。

4.1 目标坐标解算

4.1.1 计算目标相对方位

如前文所述，可以通过 yolov5 神经网络模型检测到目标，此时获取的是目标在机器人摄像头视野里面的方位，包括目标的范围和其中心点的位置。此时可以根据摄像头的水平视野范围和分辨率解算得到目标相对机器人的方向。

代码 4.1 根据摄像头位置计算目标相对方向

```
1
2 def camera_to_lidar_angle(self, object_x, fov_camera
   =1.02974, resolution_width=1920):
3     # 计算每个像素的视角宽度(弧度)
4     fov_per_pixel = fov_camera / resolution_width
5     # 计算图像中心位置
6     image_center = resolution_width / 2
7     # 计算激光雷达的角度(弧度)
8     angle = (object_x - image_center) * fov_per_pixel
9
10    print(f"角度{angle}, object{object_x}")
11    laser_angle = angle
12    return laser_angle
```

需要注意的是，由于神经网络模型的性能限制，有时会产生一定的误识别，因此采取置信度过滤的策略。具体而言，当检测到目标时，首先判断其置信度是否满足最低要求。置信度同时也作为识别到多个目标时的优先级判断标注，若视野中同时检测到多个目标，则在满足条件的目标中选择置信度最高的一项。

4.1.2 计算目标距离

计算目标相对机器人的方位目的在于调用激光雷达对应方位的数据，以解算出目标的距离。参考以下的激光雷达话题数据格式（表 4-1），当获取到目标相对机器人的方位后，即可通过 `ranges` 数组对应的角度数据得到目标的相对距离。

表 4-1 LaserScan 话题的激光扫描数据格式

消息字段	含义
<code>header</code>	消息的时间戳和坐标系信息
<code>angle_min</code>	激光扫描的起始角度，单位为弧度
<code>angle_max</code>	激光扫描的终止角度，单位为弧度
<code>angle_increment</code>	每个扫描点之间的角度增量，单位为弧度
<code>time_increment</code>	每个扫描点之间的时间增量，单位为秒
<code>scan_time</code>	完成一次扫描所需的时间，单位为秒
<code>range_min</code>	激光雷达的最小有效范围，单位为米
<code>range_max</code>	激光雷达的最大有效范围，单位为米
<code>ranges</code>	激光扫描每个角度对应的距离数据，单位为米
<code>intensities</code>	激光扫描每个角度对应的强度数据（没有时可为空）

4.1.3 解算目标坐标

当测得目标与机器人的相对方向和距离，就得到了目标相对机器人的方位（代码 4-2），此时再结合当前时刻的机器人位姿，即可解算得到目标在全局地图上的坐标代码（4-3）。

代码 4.2 计算目标相对机器人的方位

```
1
2   angle_index = int((target_angle - scan_msg.angle_min) /
3   scan_msg.angle_increment)
4   if 0 <= angle_index < len(scan_msg.ranges):
5       distance = scan_msg.ranges[angle_index]
6       #判断识别到的目标是否在激光雷达范围之外
7       if distance != float('inf') and distance < 2.8:
```

```

7         print(f"距离:{distance}")
8         self.target_in_laser = True
9     else:
10        self.target_in_laser = False
11        return
12    #使用三角函数计算相对位置
13    local_x = distance * math.cos(target_angle)
14    local_y = distance * math.sin(target_angle)

```

其中机器人当前的位姿数据来自于 `robot_pose` 消息。值得注意的是，由于在 ROS 中使用的位姿尤其是机器人的 `pose` 或里程计的数据等大部分时候以四元数表示，优势在于计算效率高且精确，但是四元数也存在着本身不是很直观的缺点。现在需要对角度进行加减运算的时候，不便直接操作四元数，因此需要进行将四元数转换为欧拉角的过程。

代码 4.3 计算目标相对机器人的方位

```

1
2    robot_x = robot_pose.position.x
3    robot_y = robot_pose.position.y
4    robot_theta = euler_from_quaternion([
5        robot_pose.orientation.x,
6        robot_pose.orientation.y,
7        robot_pose.orientation.z,
8        robot_pose.orientation.w
9    ])[2]
10    #计算目标在全局地图中的坐标
11    global_x = robot_x + local_x * math.cos(robot_theta) -
12               local_y * math.sin(robot_theta)
13    global_y = robot_y + local_x * math.sin(robot_theta) +
14               local_y * math.cos(robot_theta)

```

4.2 传感器数据融合优化

上文提到的流程可以计算出目标的坐标数据，但是由于机器人通常在运动过程中进行这个流程，同时目标检测、激光雷达数据读取、机器人位姿数据都存在一定的时延。当这些时间延迟叠加在计算流程中，显然会造成明显的误差。因此在流程中融合这些数据时，十分有必要通过时间戳对这些数据进行同步，以使用在目标检测时刻的真实数据。在 ROS2 中，上述话题在发布时都会具有时间戳，但是仍然有必要对其进行进一步的操作。

4.2.1 对于目标检测信息的时间戳调整

目标检测模块在检测到目标后，会通过对应的话题发布目标检测数据，此时的目标检测结果话题中带有时间戳，这个时间戳表示了该话题发布的时刻。但是事实上，神经网络在对目标进行推理检测时会耗费一段时间，例如在一段目标识别模块的打印信息中（图 4-1），注意这样一条数据：Speed: 2.1ms pre-process, 43.9ms inference, 0.2ms NMS per image at shape (1, 3, 384, 640)

以上这条信息中：

- 2.1ms pre-process：表示图像预处理耗时 2.1 毫秒。预处理包括图像的缩放、归一化等操作。
- 43.9ms inference：表示模型推理耗时 43.9 毫秒。推理是指模型对图像进行目标检测的过程。
- 0.2ms NMS：表示非极大值抑制（Non-Maximum Suppression, NMS）耗时 0.2 毫秒。NMS 用于去除重叠的检测框，保留最可能的检测结果。

这表示了在整个处理流程中，预处理耗时 2.1 毫秒，推理耗时 43.9 毫秒，NMS 耗时 0.2 毫秒。这些时间会直接导致检测信息和其他数据融合时产生误差。因此需要调整时间戳的来源，选择获取到原始图像的时刻作为对应为该信息的时间戳并进行发布，避免默认时间戳造成的延时误差。

4.2.2 时间戳队列同步

为了同步各个传感器的数据并使用时间戳进行精确同步，为此需要建立一个同步机制。最终采用的方案是建立多个消息队列，通过选取最接近的数据进行合并以达到同步的效果。

代码 4.4 传感器消息的同步队列

```
1 # 队列缓存，最大缓存长度为100条
2 # 两个机器人的激光雷达消息队列
3 self.scan_queue_robot1 = deque(maxlen=100)
4 self.scan_queue_robot2 = deque(maxlen=100)
5 # 两个机器人的视觉识别数据消息队列
```

```

jstone@jstone-virtual-machine: ~/Documents/graduation_pr...
jstone@... x jstone@... x jstone@... x jstone@j... x jstone@j... x
keyboardinterrupt
jstone@jstone-virtual-machine:~/Documents/graduation_project/1P/robot_army$ ros2
run yolov5_ros2 yolo_detect_2d --ros-args -p device:=cpu
/opt/ros/galactic/bin/ros2:6: DeprecationWarning: pkg_resources is deprecated as
an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import load_entry_point
[INFO] [1742202460.495738014] [yolov5_ros2]: Current ROS 2 distribution: galacti
c
[INFO] [1742202460.945031766] [yolov5_ros2]: Robot 0 detection result: image 1/1
: 1080x1920 (no detections)
Speed: 6.0ms pre-process, 59.0ms inference, 1.1ms NMS per image at shape (1, 3,
384, 640)
[INFO] [1742202461.008842060] [yolov5_ros2]: Robot 1 detection result: image 1/1
: 1080x1920 (no detections)
Speed: 3.9ms pre-process, 45.8ms inference, 0.2ms NMS per image at shape (1, 3,
384, 640)
[INFO] [1742202461.070156984] [yolov5_ros2]: Robot 0 detection result: image 1/1
: 1080x1920 (no detections)
Speed: 2.1ms pre-process, 43.9ms inference, 0.2ms NMS per image at shape (1, 3,
384, 640)
[INFO] [1742202461.132090550] [yolov5_ros2]: Robot 1 detection result: image 1/1
: 1080x1920 (no detections)
Speed: 2.1ms pre-process, 45.6ms inference, 0.3ms NMS per image at shape (1, 3,
384, 640)

```

图 4-1 目标识别模块运行信息

```

6 self.detection_queue_robot1 = deque(maxlen=100)
7 self.detection_queue_robot2 = deque(maxlen=100)
8 # 两个机器人的位姿消息队列
9 self.odom_queue_robot1 = deque(maxlen=100)
10 self.odom_queue_robot2 = deque(maxlen=100)
11 # 机器人分别记录的目标坐标
12 self.marked_positions_robot1 = []
13 self.marked_positions_robot2 = []

```

每个传感器的消息会先分别进入队列，由于传感器数据融合的目的是在目标被检测到时融合对应时刻的传感器数据进行坐标解算，因此当 yolo 识别数据进入队列时，会在队列中对对应的激光雷达数据和位姿数据进行匹配。

代码 4.5 当 yolo 模型检测到目标时对数据进行匹配

```

1 if not self.scan_queue_robot1 or not self.
  detection_queue_robot1 or not self.odom_queue_robot1:
2 return # 缓存不完整，无法同步
3 # 获取YOLO识别数据
4 cam_detection = self.detection_msg_robot1 # 使用已保存的
  detection_msg_robot1
5 if not cam_detection:
6 return # 如果没有YOLO数据，跳过
7 # 获取与YOLO数据时间戳接近的位姿数据
8 odom_msg = self.get_closest_message_within_time(
  cam_detection.header.stamp, self.odom_queue_robot1)
9 # 获取与YOLO数据时间戳接近的激光雷达数据
10 scan_msg = self.get_closest_message_within_time(
  cam_detection.header.stamp, self.scan_queue_robot1)
11 if scan_msg and cam_detection and odom_msg:

```

```
12 self.process_robot1(scan_msg, cam_detection, odom_msg)
```

其中，时间戳匹配的函数 `get_closest_message_within_time` 会循环查找队列找到最小差异的消息，并且可以调整时间戳差异范围，过滤不同精度的数据。

代码 4.6 当 yolo 模型检测到目标时对数据进行匹配

```
1 def get_closest_message_within_time(self, detection_stamp,
2   message_queue, max_time_diff=TIME_DIFF):
3     if len(message_queue) == 0:
4       return None
5     closest_msg = None
6     min_time_diff = float('inf')
7     for msg in message_queue:
8       # 获取消息的时间戳 (秒)
9       msg_time = msg.header.stamp.sec + msg.header.stamp.
10        nanosec / 1e9
11       # 获取YOLO检测数据的时间戳 (秒)
12       detection_time = detection_stamp.sec + detection_stamp
13        .nanosec / 1e9
14       # 计算时间差
15       time_diff = abs(msg_time - detection_time)
16       #print(f"Message Time: {msg_time}, Detection Time: {
17        detection_time}, Time Diff: {time_diff}")
18       # 如果时间差小于最大时间差，认为符合条件
19       if time_diff <= max_time_diff:
20         # 如果这是第一次符合条件的消息，或者当前消息比已找到的更接近
21         # 检测数据的时间
22         if closest_msg is None or time_diff < min_time_diff:
23           :
24           closest_msg = msg
25           min_time_diff = time_diff
26     return closest_msg
```

通过以上方法手动地完成了多个传感器数据的融合，减少了各种数据传输和推理延迟造成的解算结果偏差。

4.3 数据处理模块优化目标坐标

当机器人对目标进行检测与坐标解算时会得到目标坐标（图 4-2 右侧子图为仿真环境下目标啤酒罐的摆放位置，右侧子图是机器人在地图上发布的目标坐标标记结果），然而尽管机器人在数据融合时使用了时间戳同步的策略来提高目标坐标精度，但是受限于激光雷达与摄像头的帧率等硬件因素，最终解算得到的目标坐标仍然会存在误差。在对同一目标多帧数据的解算后，其误差通常表现为目标坐标分布为一片离散的点（图 4-2 右侧子图中展示的绿色点）。为此，有必要采

用数据处理手段对这些数据进行补偿和清洗。

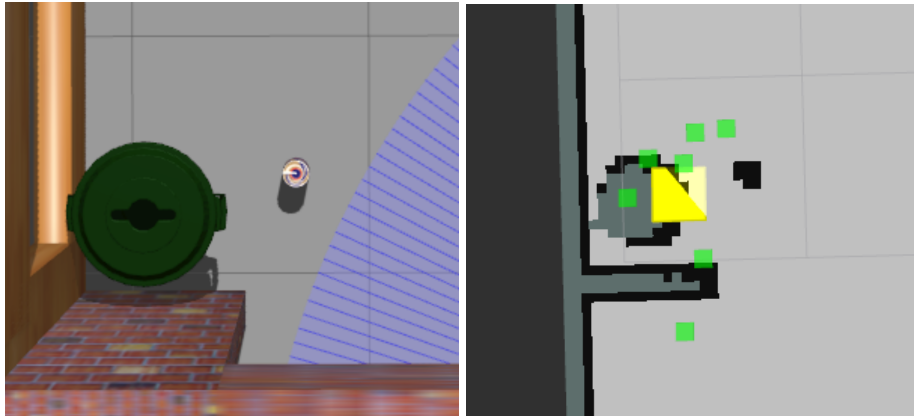


图 4-2 仿真环境中的目标与地图中的目标标记

针对离散的目标坐标分布，可以认为其大致随机分布在目标附近，因此考虑采用聚类算法对原始的目标坐标数据进行处理。因此在 `target_detect` 包下建立 `marker_fix` 模块进行数据处理，考虑到点的分布具有一定的密度，选择使用 DBSCAN 聚类算法处理点坐标数据。DBSCAN 聚类算法的优势在于可以选择参数来过滤一些由于遮挡或者偏移导致过于错误形成聚集的目标点。当机器人发布点的坐标时，会同时对全局的坐标点进行重新聚类，以实现目标地图的动态调整。这意味着目标坐标的精度会不断随着机器人对目标检测的次数提高。当完成聚类后，聚类后的目标点坐标会被发布到地图中（图 4-2 右侧子图黄色较大点）。

4.4 本章小结

本章主要介绍的是目标检测与搜索流程设计，其中核心包是 `yolov5_ros2` 和 `target_detect` 包，功能主要围绕 ROS 节点 `multirobot_map_updater` 进行，该节点会订阅来自目标检测模块的话题，当目标检测模块检测到目标时结合激光雷达数据和机器人位姿数据对目标坐标进行解算与发布，并且采取一系列措施提高检测精度（图 4-3）。

以下表格（表 4-2）展示了图中（图 4-3）每个话题的含义：

表 4-2 mulit_robot_map_updater 节点的话题列表

话题	含义
odom	机器人位姿
scan	激光雷达数据
yolo_result	目标检测结果
merge_map	多机器人融合后的地图
map_markers	识别后的原始坐标
map_markers_fix	数据处理修正后的坐标

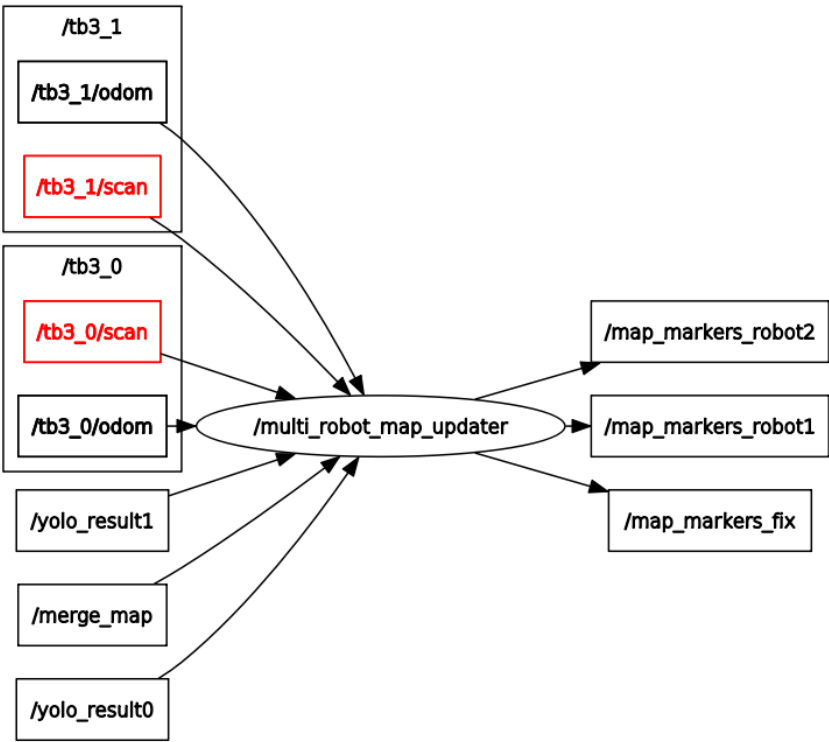


图 4-3 mulit_robot_map_updater 节点的 ROS 结构

第五章 路径规划与控制模块设计

在多机器人协同搜索系统中，需要控制多个机器人完成对空间的探索，而路径规划是确保机器人高效覆盖目标区域、避免碰撞并优化搜索效率的关键环节。本模块旨在为多个机器人提供合理的路径规划方案，使其在动态环境中能够高效协作，减少重复覆盖，并保证搜索任务的全面性和实时性。为了实现这一目标，本研究综合考虑了机器人间的避让机制、全局路径优化以及局部调整策略，确保系统在复杂环境下依然具备鲁棒性和适应性。本章将详细介绍路径规划的算法选择、实现方式及优化策略，并探讨如何结合 ROS2 的多机器人通信框架，实现高效的路径规划与调度。

5.1 模块架构

路径规划与控制模块由 multi_robot_exploration 包和 path_follow 两个包组成。主要流程包括 multi_robot_exploration 包根据机器人获取的最新地图进行路径规划，完成规划后 path_follow 功能包会订阅该路线话题并控制机器人根据路径运动，其相关节点通信如下图（图 5-1）。

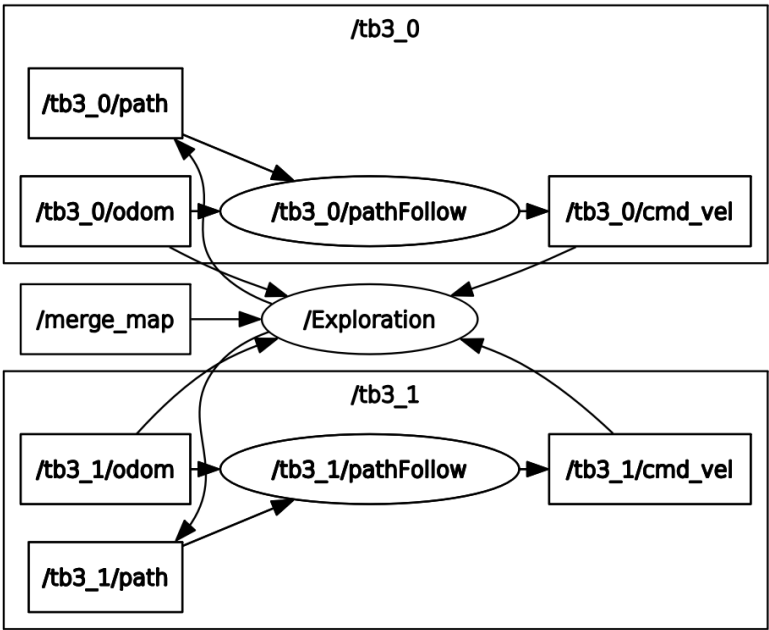


图 5-1 路径规划与控制模块节点结构

其中 /Exploration 节点属于 multi_robot_exploration 包，在订阅最新的全局地图话题 /merge_map 后会发布路径规划话题 /path，节点 /pathFollow 根据 /Exploration 节点发布的路径分别控制对应的机器人。

5.2 路径规划模块

5.2.1 边界识别与探索区域分配

模块已经获取了最新的全局地图，并且其形式为栅格地图，为完成区域探索，需要进行边界识别与探索区域分配。边界识别的核心目标是确定已知区域与未知区域的交界处，即探索的前沿区域。在栅格地图中，已知区域通常由自由空间和障碍物构成，而未知区域尚未被机器人感知。因此，需要遍历整个栅格地图，寻找满足以下条件的栅格点：

- 该点是自由空间（即机器人可以通过）。
- 该点的相邻栅格中至少有一个是未知区域。

通过遍历地图并检测四邻域（上下左右）是否存在未知区域，可以高效地提取所有边界点，形成一个边界点集合 **frontiers**。这些边界点代表了机器人需要探索的目标区域，为进一步的任务分配提供了基础。

由于边界点分布通常是不均匀的，因此单独考虑每个边界点可能导致探索路径冗余或重复覆盖。为此，采用聚类算法将相邻的边界点归为一个区域，并进一步确定最优的探索目标点。具体的工作流程包括：

- 基于广度优先搜索（BFS）进行区域划分遍历 **frontiers** 中的所有点，采用 BFS 搜索相邻的边界点，并将其聚类到同一个探索区域中。通过记录访问状态，确保每个点只被访问一次，从而形成多个不相交的探索区域 **groups**。

代码 5.1 使用 BFS 对边界点进行聚类，形成多个区域

```
1 def cluster_frontiers(frontiers):
2     """
3     使用BFS对边界点进行聚类，形成多个区域
4     :param frontiers: 边界点列表 [(x, y), (x, y), ...]
5     :return: clustered_regions (多个探索区域，每个区域是边界点的集合)
```

```

6         """
7         clustered_regions = []
8         visited = set()
9         for frontier in frontiers:
10             if frontier in visited:
11                 continue
12             # BFS 聚类
13             queue = deque([frontier])
14             region = []
15             while queue:
16                 x, y = queue.popleft()
17                 if (x, y) in visited:
18                     continue
19                 visited.add((x, y))
20                 region.append((x, y))
21                 # 搜索四邻域
22                 neighbors = [(0,1), (0,-1), (1,0), (-1,0)]
23                 for dx, dy in neighbors:
24                     nx, ny = x + dx, y + dy
25                     if (nx, ny) in frontiers and (nx, ny)
26                         not in visited:
27                         queue.append((nx, ny))
28                 clustered_regions.append(region)
29             return clustered_regions

```

- 计算探索区域的中心点（质心）对于每个探索区域，计算其所有边界点的几何均值，以获得该区域的中心点 **centroid**，作为机器人前往的目标点。该方法可以避免单点目标的局限性，使机器人能够覆盖整个区域，提高探索效率。

代码 5.2 计算每个区域的中心点

```

1     def compute_centroids(clustered_regions):
2         """
3         计算每个区域的中心点
4         :param clustered_regions: 已聚类的边界点区域
5         :return: 质心列表 [(cx, cy), (cx, cy), ...]
6         """
7         centroids = []
8         for region in clustered_regions:
9             x_coords, y_coords = zip(*region)
10            centroid = (sum(x_coords) / len(x_coords), sum
11                        (y_coords) / len(y_coords))
12            centroids.append(centroid)
13        return centroids

```

- 最优目标选择由于部分探索区域可能过小或分布零散，我们对区域大小进行筛选，选取规模最大（点数最多）的前 5 个区域，避免机器人前往过小的目标区域。机器人在前往目标时，优先选择距离最近的区域，以最小化行驶距离，提高任务执行效率。

5.2.2 进行路径规划

在确定最优目标后，需要进行路径规划以前往目标，因此需要计算机器人从当前位置到目标点的最优路径，避免障碍物，并尽可能缩短行驶距离。常见的路径规划方法包括 A* 算法和 Dijkstra 算法。在本项目中采用 A* 算法（A-star），因考虑其在计算效率与路径优化方面的良好平衡性。

为实现 A* 算法，需要在搜索过程中综合考虑当前代价（g）和预计代价（h）：

- $g(n)$: 从起点到当前点 n 的实际代价（路径长度）。
- $h(n)$: 从当前点 n 到目标点的距离。
- $f(n) = g(n) + h(n)$: 作为路径搜索的评估函数，选择 $f(n)$ 最小的点作为下一步搜索目标。

代码 5.3 实现 A* 算法

```

1  def astar_search(grid, start, goal):
2      """
3      A*路径规划
4      :param grid: 栅格地图, 0表示可行区域, 1表示障碍物
5      :param start: 机器人起始位置 (x, y)
6      :param goal: 目标位置 (x, y)
7      :return: 规划路径 (列表)
8      """
9      neighbors = [(0, 1), (0, -1), (1, 0), (-1, 0)]
10     open_set = []
11     heapq.heappush(open_set, (0, start)) # (优先级, 位置)
12     came_from = {}
13     g_score = {start: 0}
14     f_score = {start: heuristic(start, goal)}
15     while open_set:
16         _, current = heapq.heappop(open_set)
17         if current == goal:
18             path = []
19             while current in came_from:
20                 path.append(current)
21                 current = came_from[current]
22             return path[::-1] # 反转路径
23         for dx, dy in neighbors:
24             neighbor = (current[0] + dx, current[1] + dy)
25             if 0 <= neighbor[0] < len(grid[0]) and 0 <= neighbor[1] < len(grid) and grid[neighbor[1]][neighbor[0]] == 0:
26                 temp_g_score = g_score[current] + 1 # 假设每个移动步长为1

```

```

27         if neighbor not in g_score or temp_g_score <
           g_score[neighbor]:
28             came_from[neighbor] = current
29             g_score[neighbor] = temp_g_score
30             f_score[neighbor] = temp_g_score + heuristic(
               neighbor, goal)
31             heapq.heappush(open_set, (f_score[neighbor],
               neighbor))
32
33     return [] # 若未找到路径, 返回空列表

```

在使用 A* 算法完成路径规划后, 由于 A* 算法通常得到的是离散的直线段, 为了使机器人运动更加平稳, 需要使用 B 样条算法进行平滑。这样可以使路径更加平滑, 减少机器人运动抖动, 并且可以使用插值控制 `sn` 来调整路径平滑的精细程度。

与此同时, 为了避免机器人距离障碍物过近导致碰撞, 路径规划时使用的地图数据需要经过障碍物范围膨胀。通过对栅格地图上的障碍物坐标周边偏移并设置为障碍物获得膨胀后的障碍物地图, 需要注意的是, 障碍物膨胀需要设计障碍物膨胀系数, 若系数过小会失去膨胀的意义, 但是若是设置得过大则有可能导致门口等地形被彻底封闭, 因此需要调整得到合适的数值。

代码 5.4 障碍物膨胀方法

```

1     #障碍物扩展系数
2     expansion_size = 6
3
4     def costmap(data, width, height, resolution):
5         data = np.array(data).reshape(height, width)
6         wall = np.where(data == 100)
7         for i in range(-expansion_size, expansion_size + 3):
8             for j in range(-expansion_size, expansion_size + 3):
9                 if i == 0 and j == 0:
10                     continue
11                 x = wall[0] + i
12                 y = wall[1] + j
13                 x = np.clip(x, 0, height - 1)
14                 y = np.clip(y, 0, width - 1)
15                 data[x, y] = 100 # 直接标记为障碍物, 无需乘以分辨率
16     return data

```

作为多机器人协同系统, 路径规划时的重要任务还包括避免机器人碰撞。在路径规划过程中, 使用的策略是将双方机器人当前的坐标与路径设置为障碍物, 这样在自动路径规划时就会避开对方行进的路线从而避免碰撞。

5.2.3 机器人请求路径规划

机器人会在运行过程中，根据目前的不同条件向路径规划模块请求新的路径规划，以适应更加复杂的运行情况：

- 初始探索阶段:

当节点刚启动时，全局路径标志 `TB_PATHF` 初始值为 0。此时机器人 0 (`tb3_0`) 和机器人 1 (`tb3_1`) 会立即触发首次路径请求。这是为了生成初始探索路径。

- 机器人停止移动时:

当机器人的 `cmd_vel`（控制速度）的线速度和角速度均为 0 时，表示机器人可能到达目标点或遇到障碍物无法前进，系统会重新计算新路径并打印请求消息。

- 路径规划失败后:

如果某次路径规划失败（如 `exploration` 返回 `f=-1`），路径标志会被设为 -1，此类情况如偶然的线程阻塞导致的路径规划失败，此时当机器人重新进入可探索状态且机器人停止时，机器人就会重新请求路线。

这些条件针对了机器人遇到的各种场景，包括系统刚启动时规划初始路线、机器人完成当前路径后停止或机器人卡在障碍物前无法移动等情况，使系统具有更好的鲁棒性。

5.3 根据路径控制机器人

当路径规划模块完成规划后会发布机器人路径话题，此时需要控制机器人跟随规划的路径完成运动，将发布的路径消息转换为机器人的运动控制命令，最终完成对机器人运动的控制。

5.3.1 路径控制算法

为实现一个基于 ROS2 的路径跟随节点，关键在于采用 Pure Pursuit 算法控制机器人沿指定路径移动。Pure Pursuit 算法的核心在于机器人通过不断寻找路径上的目标点并调整方向朝向该点前进，从而实现路径跟随。首先，机器人从里程计或传感器获取当前位置 (x, y) 和朝向 θ ，然后在路径上寻找一个距离机器人当前位置超过设定前视距离的最近点 (x_t, y_t) ，作为目标点。接着，计算机器人当前坐标与目标点之间的目标方向角：

$$\theta_t = \text{atan2}(y_t - y, x_t - x)$$

并计算航向误差：

$$\Delta\theta = \theta_t - \theta$$

确保其范围在 $(-\pi, \pi]$ 内。随后，根据误差计算线速度 v （通常设定为固定值或根据路径曲率调整）和角速度 w ，其中 w 受最大角速度 $\omega_{\max}$ 限制，以确保平稳转向：

$$w = \max(-\omega_{\max}, \min(\Delta\theta, \omega_{\max}))$$

最终，机器人根据 v 和 w 进行运动，并不断重复上述步骤，直到顺利抵达目标位置，实现路径跟随。

5.3.2 路径跟随的实现

首先，pathFollower 节点订阅 /path 话题，接收并解析路径数据，同时订阅 /odom 话题，获取机器人当前位置和朝向，并发布 /cmd_vel 话题，向机器人发送速度指令，同时在 /visual_path 话题上发布可视化路径数据。

当路径信息到达时，get_path 方法解析路径数据，并启动 follow_path 线程，持续控制机器人运动。在 follow_path 过程中，机器人不断更新自身位姿，并调用 pure_pursuit 计算合适的线速度和角速度，确保沿着路径行驶，并在接近终点时停止。在 pure_pursuit 方法中，机器人寻找前视点（超过 lookahead_distance 的最近

路径点)，计算目标方向，并通过调整角速度 w 控制转向，避免急转弯，确保平稳运动。

与此同时，`odom_callback` 订阅 `/odom` 话题，将机器人位姿信息实时更新到变量中，为路径跟随提供数据支持。该代码适用于 TurtleBot3 或其他支持 `/cmd_vel` 速度控制的移动机器人，可在 Gazebo 仿真或实际机器人上运行，实现路径跟随功能。

第六章 系统的仿真与测试

上文介绍了多机器人协同搜索的关键模块与技术方案，本章将介绍对此系统的仿真与测试效果。将综合验证系统的控制、搜索以及路线规划等能力。

6.1 仿真环境

6.1.1 主要运行环境

仿真环境的基本情况包括硬件配置和软件环境（表 6-1），由于 gazebo 仿真环境涉及较大量的图像处理，对硬件性能和软件版本有一定需求，因此仿真环境的配置是值得关注的问题。

表 6-1 主要运行环境配置

项目	参数
内存	16GB
处理器	1th Gen Intel® Core™ i7-11800H @ 2.30GHz × 16
显卡	NVIDIA GeForce RTX 3060 Laptop GPU
系统	Ubuntu 20.04.6 LTS
Gazebo 版本	version 11.15.1
ROS 版本	ros2-galactic

在良好的配置下 gazebo 可以提高物理仿真参数以提高仿真的效果，使其更加接近实用条件，否则则有可能出现意料之外的错误效果。

6.1.2 主要仿真参数

对系统进行仿真时需要定义机器人参数，环境参数等，以求模拟接近真实环境。首先搭建了机器人进行搜索的地图，模拟现实环境下的房屋环境（图 6-1），包括 6 个房间。其中有书架、桌子、垃圾桶、书架、啤酒罐等物品，所有材质都具

有对应的纹理，例如木纹、大理石、塑料、红砖墙等，其中房屋结构有门、窗、走廊等。该房屋环境中共放置了 5 个啤酒罐模型，作为机器人搜索的目标。

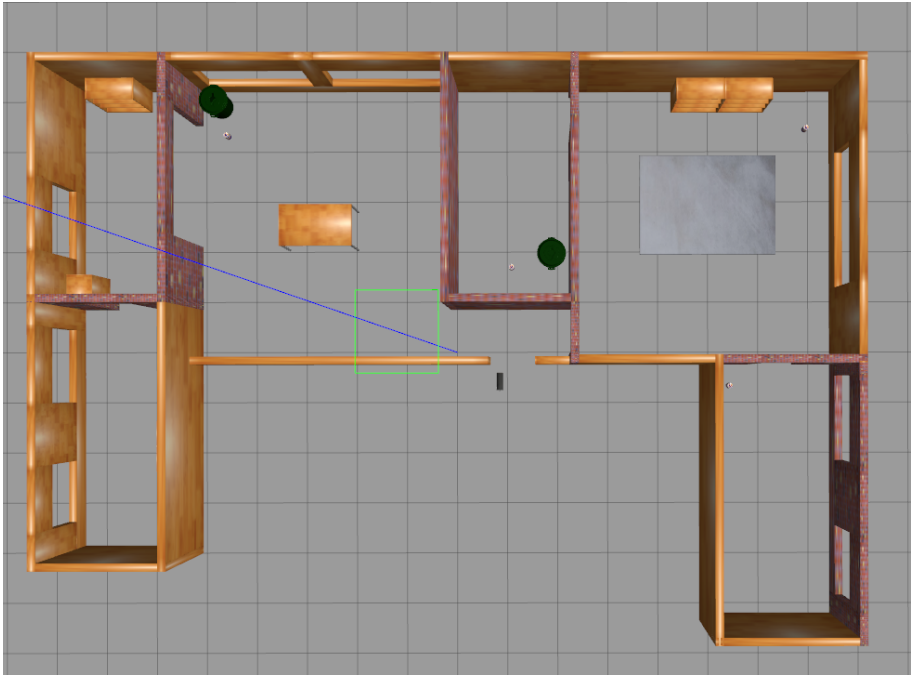


图 6-1 仿真房屋环境搭建

在仿真环境中需要设置物理引擎参数（表 6-2），物理引擎参数会决定仿真的效果和性能。本项目在硬件限制条件下尽可能调整 `real_time_update_rate` 和 `max_step_size` 以提高时间准确性。

表 6-2 Gazebo 物理引擎参数设置

参数	含义	值
<code>real_time_update_rate</code>	仿真每秒更新次数	1000.0
<code>max_step_size</code>	每步最大仿真时间步长（秒）	0.001
<code>real_time_factor</code>	仿真时间与真实时间的比例	1
<code>type</code>	ODE 求解器类型	quick
<code>iters</code>	求解器最大迭代次数	150
<code>sor</code>	超松弛参数（加速收敛）	1.4
<code>use_dynamic_moi_rescaling</code>	是否启用动态惯性矩缩放	1
<code>cfm</code>	约束柔性参数（降低约束振荡）	0.00001
<code>erp</code>	误差恢复参数（影响刚体恢复速度）	0.2
<code>contact_max_correcting_vel</code>	最大接触点修正速度	2000.0
<code>contact_surface_layer</code>	接触表面层厚度（影响碰撞检测精度）	0.01

6.2 在仿真环境下运行系统

完成环境搭建后在仿真房屋的两个不同房间里分别放入 `turtlebot_waffle` 机器人（图 6-2），其中蓝色部分指示机器人激光雷达范围，白色线条指示机器人摄像头视角范围。

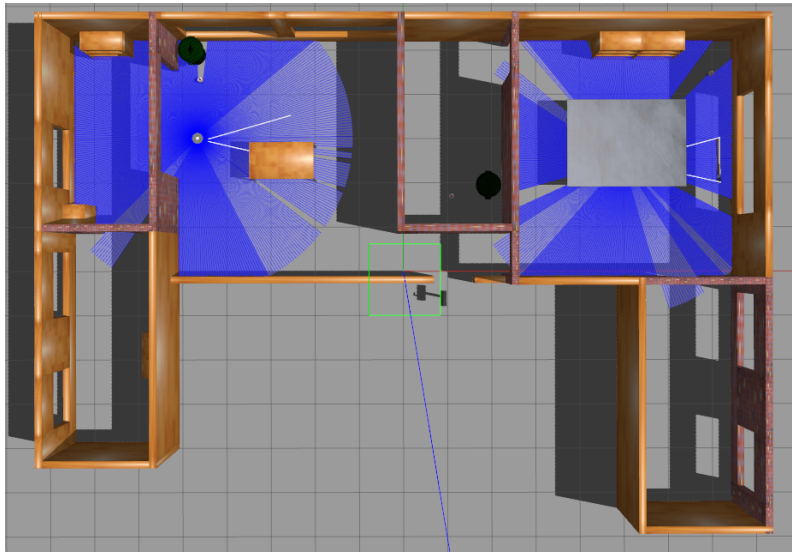


图 6-2 在仿真房屋中放入机器人

运行地图融合模块，可以在 `rviz` 中查看当前根据两个机器人激光雷达获取得到的融合地图（图 6-3）。

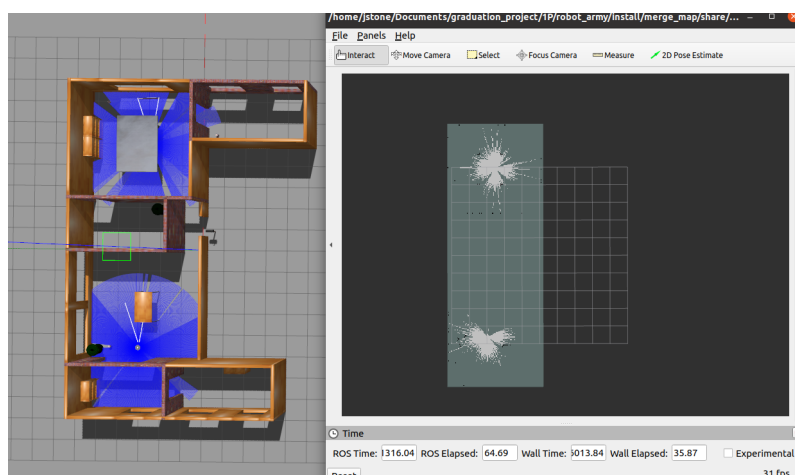


图 6-3 地图融合模块

运行 `yolo` 目标检测，可以看到 `yolov5` 开始对目标进行检测（图 6-4）。

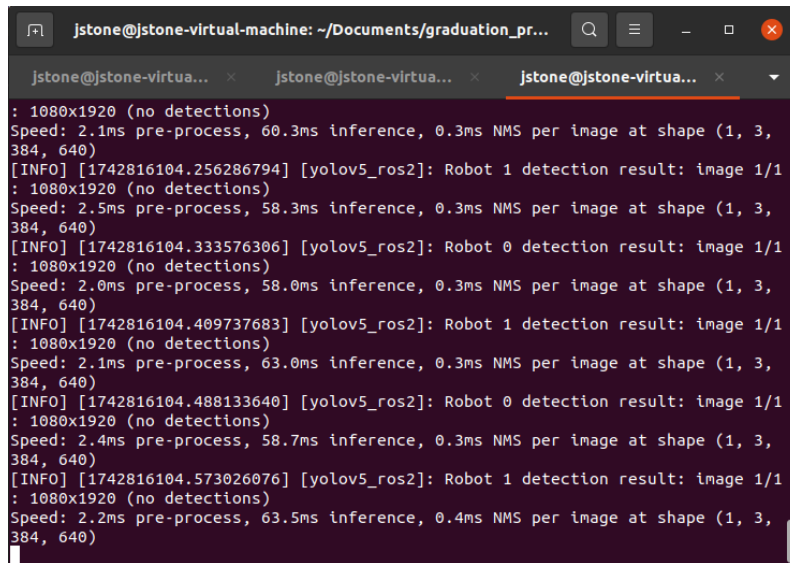


图 6-4 目标检测模块运行信息

运行目标搜索与定位模块，rviz 中将会显示三个窗口，分别展示两个机器人的摄像头画面和标记了主要信息的地图（图 6-5）。其中机器人的摄像头画面经过目标检测模块的标记，带有目标检测框信息（图 6-6）。地图展示窗口中的标记包括目标坐标标记（图 6-7）和规划路径标记，两个机器人的路径和搜索目标坐标分别标记为红色和绿色（图 6-8）以便区分。其中较大的亮黄色点即指示数据处理后的目标位置（图 6-9）。

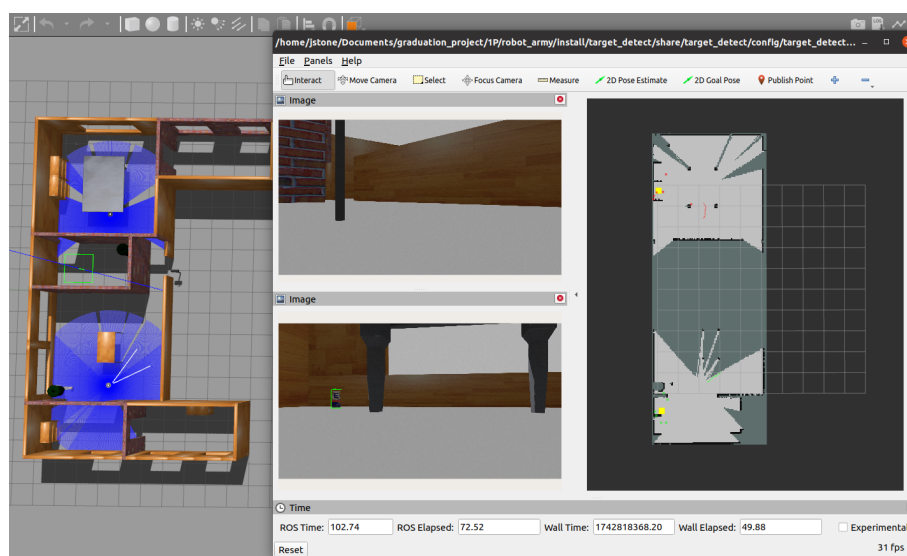


图 6-5 目标搜索与定位显示界面



图 6-6 带有目标检测框的图像

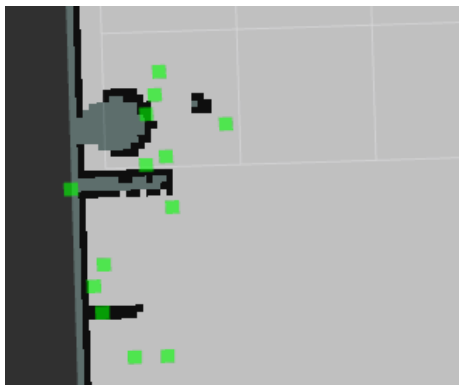


图 6-7 原始目标坐标点



图 6-8 红绿色规划路线

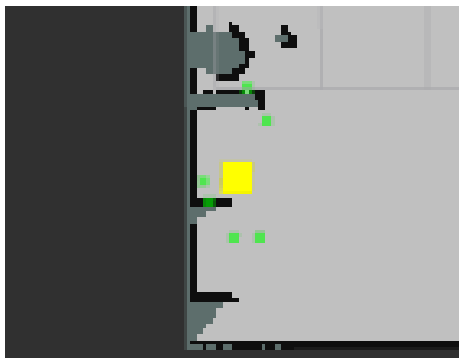


图 6-9 数据处理后坐标

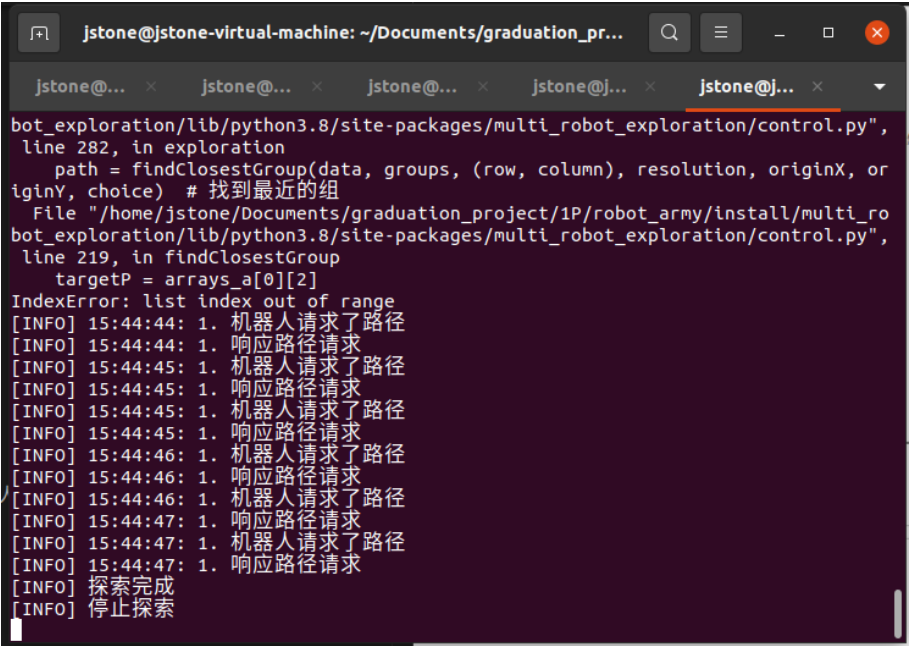
运行路径规划与控制模块，可以看见机器人开始请求路径并且模块响应，然后机器人开始运动。

```
jstone@jstone-virtual-machine: ~/Documents/graduation_pr...
jstone@... x jstone@... x jstone@... x jstone@j... x
jstone@jstone-virtual-machine:~/Documents/graduation_project/1
run multi_robot_exploration control
/opt/ros/galactic/bin/ros2:6: DeprecationWarning: pkg_resources
an API. See https://setuptools.pypa.io/en/latest/pkg_resource
from pkg_resources import load_entry_point
[INFO] 探索开始
[INFO] 19:38:27: 1. 机器人请求了路径
[INFO] 19:38:27: 2. 机器人请求了路径
[INFO] 19:38:27: 2. 响应路径请求
[INFO] 19:38:28: 1. 响应路径请求
```

图 6-10 路径规划模块响应请求

6.3 系统测试

在仿真环境下运行系统后，机器人逐渐完成对该环境的搜索（图 6-11），在所有边界被探索完成后，路径规划与运动控制模块会发出探索完毕的信息。最终得到一张该环境的完整地图，并带有目标的坐标（图 6-11）。



```
jstone@jstone-virtual-machine: ~/Documents/graduation_pr...
jstone@... x jstone@... x jstone@... x jstone@j... x jstone@j... x
bot_exploration/lib/python3.8/site-packages/multi_robot_exploration/control.py",
line 282, in exploration
    path = findClosestGroup(data, groups, (row, column), resolution, originX, or
iginY, choice) # 找到最近的组
File "/home/jstone/Documents/graduation_project/1P/robot_army/install/multi_ro
bot_exploration/lib/python3.8/site-packages/multi_robot_exploration/control.py",
line 219, in findClosestGroup
    targetP = arrays_a[0][2]
IndexError: list index out of range
[INFO] 15:44:44: 1. 机器人请求了路径
[INFO] 15:44:44: 1. 响应路径请求
[INFO] 15:44:45: 1. 机器人请求了路径
[INFO] 15:44:45: 1. 响应路径请求
[INFO] 15:44:45: 1. 机器人请求了路径
[INFO] 15:44:45: 1. 响应路径请求
[INFO] 15:44:46: 1. 机器人请求了路径
[INFO] 15:44:46: 1. 响应路径请求
[INFO] 15:44:46: 1. 机器人请求了路径
[INFO] 15:44:46: 1. 响应路径请求
[INFO] 15:44:47: 1. 机器人请求了路径
[INFO] 15:44:47: 1. 响应路径请求
[INFO] 15:44:47: 1. 机器人请求了路径
[INFO] 15:44:47: 1. 响应路径请求
[INFO] 探索完成
[INFO] 停止探索
```

图 6-11 路径规划模块完成探索

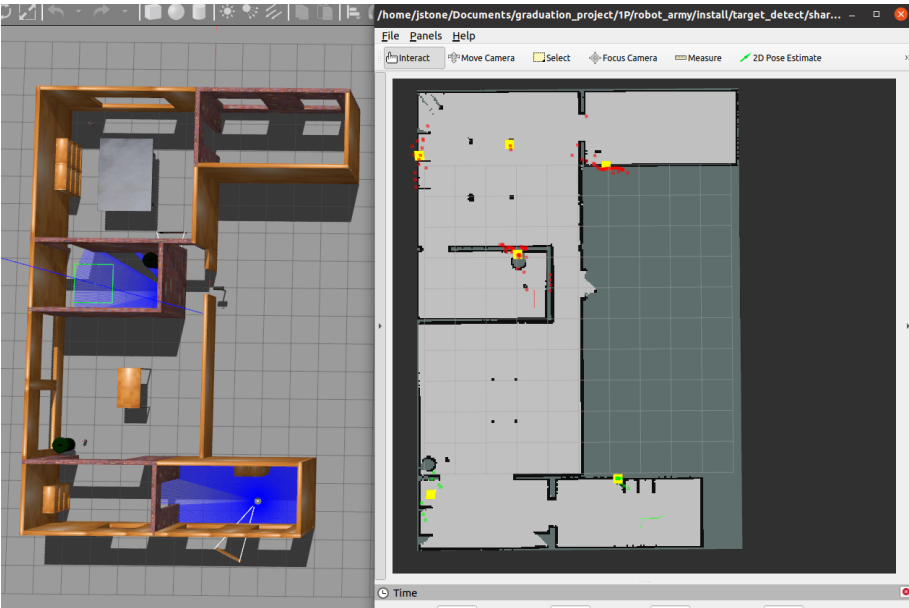


图 6-12 运行完毕得到目标地图

观察该地图中的黄色目标标记，与仿真环境中的目标分布相对比，可见搜索结果与实际情况基本对应，找出了全部啤酒罐目标，数据处理后的目标坐标与真实位置平均偏差不到 0.7 米。但是也可以发现出现了一个多余的误报目标坐标，推测是由于环境中桌腿阻挡导致的激光雷达数据异常。除此之外，系统建立的地图完整清晰，机器人协同运行流畅无冲突。测试时，由 15 时 36 分 53 机器人开始请求路径到 15 时 44 分 47 秒完成搜索，完成探索共用时 7 分 54 秒，完成了对约 90 平方米的房屋环境搜索。至此可以认为，系统的全部功能已经实现。

第七章 总结与展望

7.1 总结

本论文围绕基于 ROS 的多机器人协同搜索系统的设计与实现展开研究，针对传统单机器人系统在覆盖范围、响应速度和环境适应性方面的不足，设计了一种基于 ROS2 的分布式多机器人协同搜索系统。通过整合 YOLOv5 目标检测模型、传感器数据融合方案和改进的路径规划算法，构建了具备协作能力的多机器人搜索系统。主要涉及内容包括：

- 多机器人协同架构：基于 ROS2 Galactic 的机器人操作系统，通过模块化设计支持多机器人实时数据交互。采用地图融合模块整合多机器人局部地图，构建全局环境模型，显著提升了系统的扩展性和鲁棒性。
- 目标检测优化：通过 Gazebo 仿真环境采集数据并训练轻量化 YOLOv5s 模型，结合时间戳同步策略和 DBSCAN 聚类算法，有效改善传感器数据延迟与坐标离散问题。实验表明，目标定位平均误差较小，且能过滤大部分误检干扰。
- 协同路径规划：提出基于 A* 算法与边界聚类的动态路径规划方法，结合障碍物膨胀策略和机器人避碰机制，实现了多机器人高效探索与协同避障。仿真测试中，双机器人系统在 7 分 54 秒内完成 90 平方米复杂环境的搜索任务，验证了算法的有效性。
- 全流程验证：通过 Gazebo 与 RViz 搭建仿真测试平台，完整验证了从环境建模、目标检测到协同搜索的全流程功能。系统在动态环境适应性和任务完成效率方面效果显著，为实际应用提供了可靠的技术参考。

7.2 展望

尽管本系统在仿真环境下取得了预期成果，但仍存在很多改进方向：

- 实际场景适配：当前实验基于 Gazebo 仿真环境，物理引擎参数与真实环境存在差异。未来可在实体机器人平台上进一步验证，优化传感器噪声模型和运动控制精度。
- 算法性能提升：目标检测模块的误报率仍需降低，可通过引入多模态传感器（如热成像、深度相机）增强环境感知能力。路径规划算法可融合强化学习技术，实现动态任务分配与实时避障优化。
- 系统扩展性增强：现有架构支持双机器人协作，但大规模机器人集群的通信负载和决策效率尚未测试。后续可研究分布式共识算法等支持更多机器人协作的方案。
- 应用场景拓展：本系统目前针对较为简单的室内环境，未来可引入更多模块，扩展至安防巡检、灾害救援等复杂场景。同时，结合通信技术与物联网技术等，可进一步提升系统的实时性与部署灵活性。

本研究为多机器人协同搜索提供了一种经过仿真验证的技术方案，其模块化设计便于后续迭代。随着人工智能与机器人技术的深度融合，相信多机器人系统将在更多领域展现其智能化、集群化的优势，更好地被用于各种场景和需求。

参考文献

致 谢

本项目的顺利完成离不开冯刚导师的悉心指导，同时感谢华南师范大学几年来为我提供的良好平台。感谢华南师范大学 PIONEER 机器人战队训练了我的机器人相关技术能力，影响了我选择的技术方向，提高了我的实践能力与钻研精神。在此感谢我的父母和朋友以及所有关心与支持我的人，他们使我的学习生活充满阳光与热情。在未来的学习生活中，我将更加努力，积极进取，并且希望有朝一日能帮助更多的人。

